1.

```python
#Homework 1 - Shopping Cart Simulation
total_cost = 0
print("Enter item price and quantity. Enter -1 for price or quantity to finish.\n")
while True:
    try:
        price = float(input("Enter item price: $"))
        if price == -1:
            break
        quantity = int(input("Enter item quantity: "))
        if quantity == -1:
            break
        total_cost += price * quantity

    except ValueError:
        print("Invalid input. Please enter numeric values.")
# apply sales tax
sales_tax_rate = 0.0625
taxed_total = total_cost * (1 + sales_tax_rate)
# apply discount if applicable
if total_cost > 50:
    final_total = taxed_total - 10
    print("\nYou qualified for a $10 discount.")
else:
    final_total = taxed_total
# final amount
print(f"\nFinal total after tax and any discounts: ${final_total:.2f}")

```

```
A:\Python\python.exe
Enter item price and quantity. Enter -1 for pric
e or quantity to finish.

Enter item price: $12
Enter item quantity: 2
Enter item price: $30
Enter item quantity: 1
Enter item price: $50
Enter item quantity: 2
Enter item price: $-1

You qualified for a $10 discount.

Final total after tax and any discounts: $153.62
Press any key to continue . . .
```

2.

PalindromeChecker.py

```python
#Number 2 - Palindrome Checker
while True:
    num = int(input("Enter an integer (-1 to quit): "))
    if num == -1:
        print("Goodbye!")
        break

    original = num
    reversed_num = 0

    while num > 0:
        digit = num % 10
        reversed_num = reversed_num * 10 + digit
        num = num // 10

    if original == reversed_num:
        print("The number is a palindrome.\n")
    else:
        print("The number is NOT a palindrome.\n")

```

```
A:\Python\python.exe
Enter an integer (-1 to quit): 1234
The number is NOT a palindrome.

Enter an integer (-1 to quit): 12321
The number is a palindrome.

Enter an integer (-1 to quit): 4421
The number is NOT a palindrome.

Enter an integer (-1 to quit): 4444
The number is a palindrome.

Enter an integer (-1 to quit): -1
Goodbye!
Press any key to continue . . .
```

3.

```python
#Homework 3 - Guess Number Game

import random

while True:
    secret = random.randint(0, 16383)
    print("\nGuess my integer number between 0 and 16383, i

    while True:
        guess = int(input("Your guess: "))

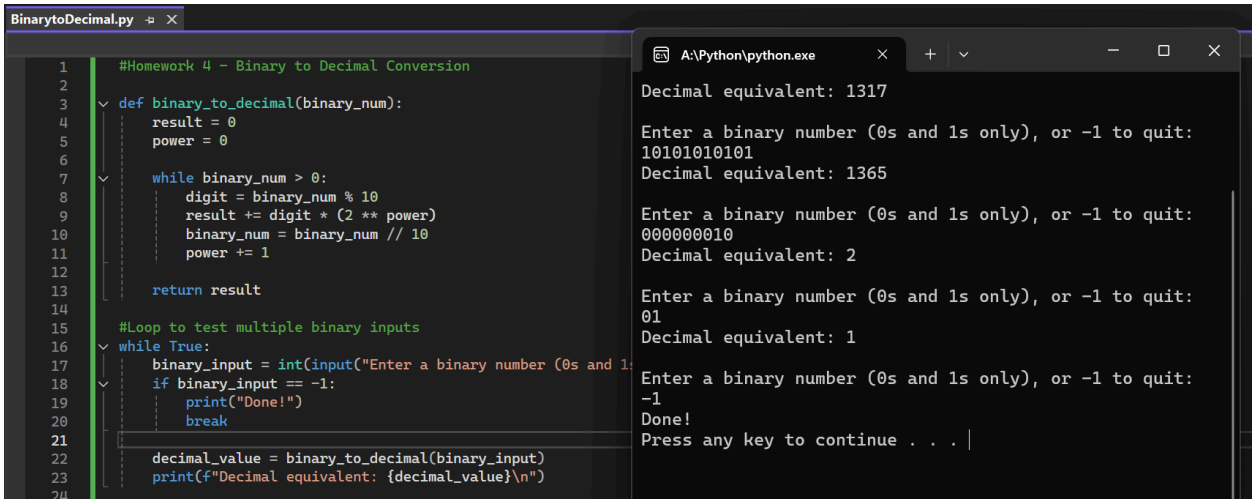
        if guess < secret:
            print("Too low. Try again.")
        elif guess > secret:
            print("Too high. Try again.")
        else:
            print("Congratulations. You guessed the number!
            break

    play_again = input("Do you want to play again? (yes/no)
    if play_again != "yes":
        print("Thanks for playing!")
        break

```

```
A:\Python\python.exe
Guess my integer number between 0 and 16383, inclusive, with t
he fewest possible guesses:
Your guess: 1122
Too low. Try again.
Your guess: 8000
Too low. Try again.
Your guess: 14000
Too high. Try again.
Your guess: 10000
Too high. Try again.
Your guess: 9000
Too high. Try again.
Your guess: 8500
Too high. Try again.
Your guess: 8200
Too low. Try again.
Your guess: 8400
Too high. Try again.
Your guess: 8300
Too high. Try again.
```

4.



BinarytoDecimal.py

```python
#Homework 4 - Binary to Decimal Conversion

def binary_to_decimal(binary_num):
    result = 0
    power = 0

    while binary_num > 0:
        digit = binary_num % 10
        result += digit * (2 ** power)
        binary_num = binary_num // 10
        power += 1

    return result

#Loop to test multiple binary inputs
while True:
    binary_input = int(input("Enter a binary number (0s and 1
    if binary_input == -1:
        print("Done!")
        break

    decimal_value = binary_to_decimal(binary_input)
    print(f"Decimal equivalent: {decimal_value}\n")
```

```
A:\Python\python.exe
Decimal equivalent: 1317

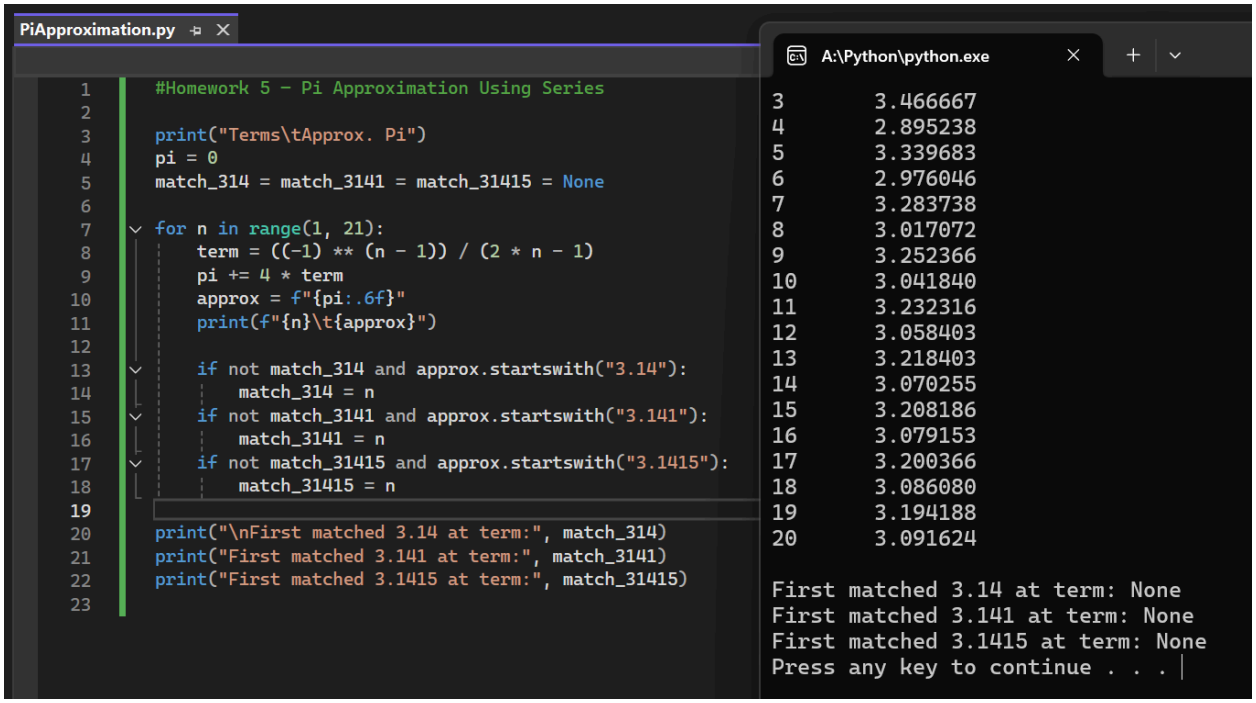
Enter a binary number (0s and 1s only), or -1 to quit:
10101010101
Decimal equivalent: 1365

Enter a binary number (0s and 1s only), or -1 to quit:
000000010
Decimal equivalent: 2

Enter a binary number (0s and 1s only), or -1 to quit:
01
Decimal equivalent: 1

Enter a binary number (0s and 1s only), or -1 to quit:
-1
Done!
Press any key to continue . . .
```

5.



PiApproximation.py

```python
#Homework 5 - Pi Approximation Using Series

print("Terms\tApprox. Pi")
pi = 0
match_314 = match_3141 = match_31415 = None

for n in range(1, 21):
    term = ((-1) ** (n - 1)) / (2 * n - 1)
    pi += 4 * term
    approx = f"{pi:.6f}"
    print(f"{n}\t{approx}")

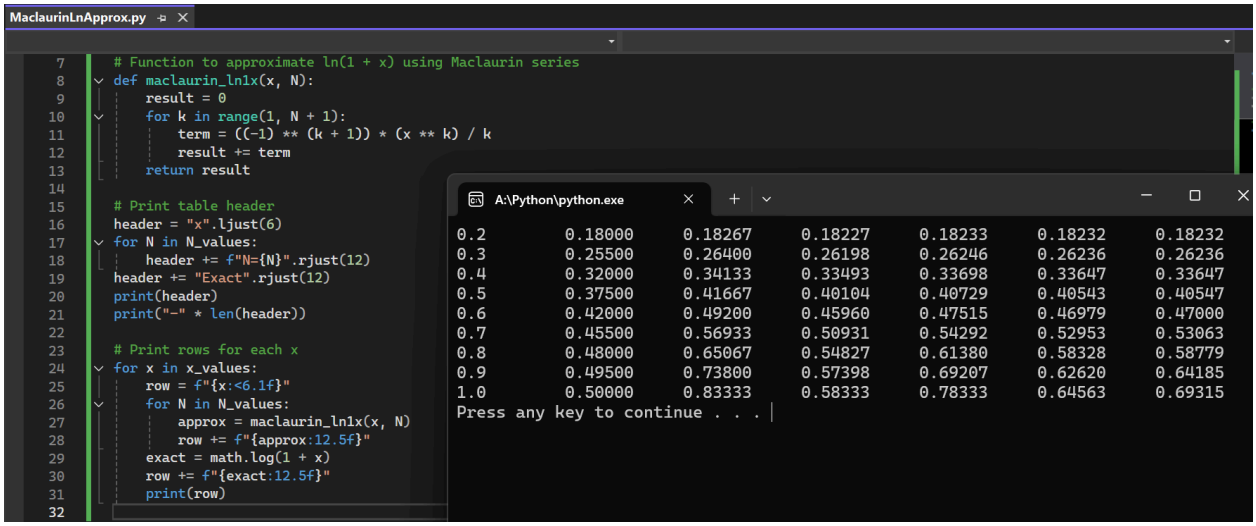
    if not match_314 and approx.startswith("3.14"):
        match_314 = n
    if not match_3141 and approx.startswith("3.141"):
        match_3141 = n
    if not match_31415 and approx.startswith("3.1415"):
        match_31415 = n

print("\nFirst matched 3.14 at term:", match_314)
print("First matched 3.141 at term:", match_3141)
print("First matched 3.1415 at term:", match_31415)
```

```
A:\Python\python.exe
3       3.466667
4       2.895238
5       3.339683
6       2.976046
7       3.283738
8       3.017072
9       3.252366
10      3.041840
11      3.232316
12      3.058403
13      3.218403
14      3.070255
15      3.208186
16      3.079153
17      3.200366
18      3.086080
19      3.194188
20      3.091624

First matched 3.14 at term: None
First matched 3.141 at term: None
First matched 3.1415 at term: None
Press any key to continue . . .
```

6.


MaclaurinLnApprox.py
Function to approximate ln(1 + x) using Maclaurin series
def maclaurin_ln1x(x, N):
result = 0
for k in range(1, N + 1):
term = ((-1) ** (k + 1)) * (x ** k) / k
result += term
return result
Print table header
header = "x".ljust(6)
for N in N_values:
header += f"N={N}".rjust(12)
header += "Exact".rjust(12)
print(header)
print("-" * len(header))
Print rows for each x
for x in x_values:
row = f"{x:<6.1f}"
for N in N_values:
approx = maclaurin_ln1x(x, N)
row += f"{approx:12.5f}"
exact = math.log(1 + x)
row += f"{exact:12.5f}"
print(row)
A:\Python\python.exe
0.2 0.18000 0.18267 0.18227 0.18233 0.18232 0.18232
0.3 0.25500 0.26400 0.26198 0.26246 0.26236 0.26236
0.4 0.32000 0.34133 0.33493 0.33698 0.33647 0.33647
0.5 0.37500 0.41667 0.40104 0.40729 0.40543 0.40547
0.6 0.42000 0.49200 0.45960 0.47515 0.46979 0.47000
0.7 0.45500 0.56933 0.50931 0.54292 0.52953 0.53063
0.8 0.48000 0.65067 0.54827 0.61380 0.58328 0.58779
0.9 0.49500 0.73800 0.57398 0.69207 0.62620 0.64185
1.0 0.50000 0.83333 0.58333 0.78333 0.64563 0.69315
Press any key to continue . . .